



DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Microkernel RISC-V

Adaptación del TP3 *Crystal Campus* de x86 (IA-32) a RISC-V (rv64)

20 de abril de 2026

Organización del Computador II
TP Final

Integrante	LU	Correo electrónico
Joaquín Romera	183/16	joakromera@gmail.com



Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

Ciudad Universitaria - (Pabellón I/Planta Baja)

Intendente Güiraldes 2160 - C1428EGA

Ciudad Autónoma de Buenos Aires - Rep. Argentina

Tel/Fax: (54 11) 4576-3359

<http://www.fcen.uba.ar>

Resumen

Este informe documenta la adaptación del TP3 *Crystal Campus* de Organización del Computador II, originalmente implementado sobre x86 (IA-32) en modo protegido y corrido bajo Bochs, a la arquitectura RISC-V de 64 bits (*rv64imac*) sobre la máquina *virt* de QEMU. El objetivo es preservar la mecánica de juego y el esqueleto de ejercicios del TP original, reemplazando los mecanismos propios de IA-32 (GDT, IDT, TSS, PIC 8259A, PIT 8254, teclado PS/2, memoria de video VGA en 0xB8000, paginación de dos niveles con CR0/CR3) por sus equivalentes nativos de RISC-V (Sv39, *stvec*, *satp*, CLINT/ACLINT, UART 16550, modos M/S/U). El kernel corre sin firmware previo (QEMU `-bios none`): arranca en **M-mode**, se configura él mismo (PMP, delegación de traps, timer) y desciende a **S-mode**, donde vive el grueso del kernel; las tareas mineras corren en **U-mode**. La arquitectura distintiva del TP (tener el código de cada minero físicamente alojado en la celda del tablero donde se encuentra) se preserva sobre Sv39, manteniendo la noción de “tablero como memoria”.

Índice

1. Introducción	4
2. Entorno y herramientas	5
2.1. De Bochs a QEMU virt	5
2.2. Toolchain y flags	5
3. Mapeo general x86 → RISC-V	6
4. Ejercicio 1 — Arranque y paso a modo supervisor	7
4.1. Qué hacía el TP x86	7
4.2. Equivalente en RISC-V	7
4.3. riscv/entry.S	7
4.4. riscv/machine.S: mini firmware en M-mode	8
4.5. “Segmento de pantalla” y limpieza inicial	8
5. Ejercicio 2 — Manejo de excepciones: IDT → stvec	9
5.1. Qué hacía el TP x86	9
5.2. Equivalente en RISC-V	9
5.3. Guardado de contexto: TrapFrame y trap.S	9
5.4. Probar una excepción	10
6. Ejercicio 3 — Reloj, teclado e interrupción 47	11
6.1. Reloj: PIT + PIC → MTIP + SSIP	11
6.2. Teclado: IRQ1 + scancodes → UART polling	12
6.3. int 47 (x86) → ecall (RISC-V)	12
7. Ejercicio 4 — Paginación del kernel	13
7.1. Qué hacía el TP x86	13
7.2. Equivalente en RISC-V: Sv39	13
7.3. Identity mapping del kernel	13
7.4. UART como “MMIO del video”	13
7.5. Activación de paginación y PMP	14
8. Ejercicio 5 — MMU de tareas	15
8.1. Qué hacía el TP x86	15
8.2. Equivalente en RISC-V	15
8.3. Allocator del kernel	15
8.4. “Tablero como memoria”	15
8.5. Construcción de la page-table de tarea	16
8.6. mmu_task_move: movimiento del minero	16
8.7. ASIDs	17
9. Ejercicio 6 — Contexto de tarea: TSS → TrapFrame	18
9.1. Qué hacía el TP x86	18
9.2. Equivalente en RISC-V	18
9.3. Inicialización de TrapFrame (equivalente de fill_tss)	18
9.4. “Salto a la tarea Idle”	19
9.5. La tarea Idle	19

10. Ejercicio 7 — Scheduler y servicios	20
10.1. Estructuras del scheduler	20
10.2. sched_nextTask	20
10.3. Despacho de ecall	21
10.4. Intercambio de tareas por cada tick	21
10.5. Atención de excepciones y desalojo	22
10.6. Modo debug	22
11. Despacho de tareas y lógica del juego	23
12. Tareas de usuario	24
13. Limitaciones y diferencias respecto al TP3 original	25
14. Build y ejecución	26
14.1. Dependencias	26
14.2. Compilación	26
14.3. Ejecución	26
14.4. Controles	26

1. Introducción

El TP3 de la materia se implementaba originalmente sobre x86 (IA-32) en modo protegido, con boot desde floppy, BIOS y ejecución bajo el simulador Bochs. Este trabajo final documenta la adaptación de ese mismo proyecto a la arquitectura RISC-V de 64 bits (rv64imac) sobre la máquina virt de QEMU.

La idea de este trabajo es preservar la mecánica de juego y el esqueleto de ejercicios del TP original, reemplazando los mecanismos específicos de IA-32 (GDT, IDT, TSS, PIC 8259A, PIT 8254, teclado PS/2, memoria de video VGA en 0xB8000, paginación de dos niveles con CR0/CR3) por sus equivalentes nativos de RISC-V. En particular, la consigna principal del TP — tener el código de cada minero *físicamente alojado* en la celda del tablero donde se encuentra y, al moverse, copiar su imagen de 8~KiB a la nueva celda y remapear TASK_CODE — se mantiene uno a uno, pero ahora implementada sobre Sv39.

El kernel corre sin firmware previo (QEMU -bios none). Esto implica que el propio kernel arranca en **modo máquina (M-mode)**, se configura a sí mismo (PMP, delegación de traps, tick del timer) y desciende luego a **modo supervisor (S-mode)**, donde vive el grueso del kernel. Las tareas mineras corren en **modo usuario (U-mode)**. Los detalles de los CSRs y modos de privilegio siguen~[2].

Este informe está organizado siguiendo la misma numeración de ejercicios del enunciado y del informe x86 original, para que la correspondencia entre ambos sea uno a uno. En cada sección se indica, primero, qué se hacía en x86 y, a continuación, qué mecanismo nativo de RISC-V lo reemplaza y dónde está implementado dentro de la carpeta riscv/ del repositorio.

2. Entorno y herramientas

2.1. De Bochs a QEMU virt

El TP original corría sobre Bochs, emulando un IBM-PC compatible con boot desde floppy y BIOS. En RISC-V esto no aplica; la opción estándar para bare-metal es la máquina virt de QEMU [7], que expone:

- Un conjunto de HARTs rv64 (este port usa uno solo).
- RAM a partir de 0x80000000.
- Un UART 16550 [10] mapeado en 0x10000000.
- CLINT (Core Local Interruptor) con mtime/mtimecmp en el rango 0x02000000 [5].
- PLIC en 0x0C000000 (no se usa en este port; el tick sale del CLINT).

Al arrancar con `-bios none`, QEMU carga el ELF del kernel y salta a 0x80000000 en M-mode, sin firmware intermedio. Esto deja al kernel la responsabilidad de todo el boot, incluyendo la programación del timer. El wiki OSDev [11] documenta en detalle esta forma de arranque y el layout de MMIO de la máquina virt.

2.2. Toolchain y flags

Se utiliza la toolchain `riscv64-unknown-elf-*` (GCC bare-metal) y `qemu-system-riscv64`. Los flags relevantes del Makefile del port:

```
ARCH_OPTS = -march=rv64imac -mabi=lp64
COMMONFLAGS = -Wall -O2 -ffreestanding -nostdlib -nostartfiles \
               -fno-stack-protector -fno-exceptions -fno-rtti \
               -fno-threadsafe-statics -fno-unwind-tables \
               -fno-asynchronous-unwind-tables -mcmodel=medany $(ARCH_OPTS)
```

El *code model* medany es necesario porque el kernel se ubica en 0x80000000, muy fuera del rango ± 2 GiB del PC que usa medlow para instrucciones PC-relativas extendidas [3]. El script del linker (`riscv/linker.ld`) fija `._ = 0x80000000` y coloca `_start` primero, de manera que el punto de entrada coincide con el reset vector de QEMU.

3. Mapeo general x86 → RISC-V

La siguiente tabla resume las equivalencias que se irán desarrollando en las secciones siguientes. Varios elementos del TP3 no tienen contraparte directa y su función la cumple otro mecanismo; se indica con “—”.

TP3 x86	RISC-V (este port)
GDT + segmentación (code/data, nivel 0/3)	— (no hay segmentos). Permisos por PTE Sv39 (U/R/W/X).
Modo real → modo protegido + A20	M-mode → S-mode vía <code>mret</code> ; PMP abierta.
CR0.PG + CR3	CSR <code>satp</code> con modo SV39 + ASID + PPN.
IDT + 255 ISR stubs	CSR <code>stvec</code> (único) + despacho en C por <code>scause</code> .
Stack separada en cambio de nivel (SS0:ESP0 en TSS)	Stack dedicada de trap (<code>trap_stack_top</code>) y salto de gp al del kernel.
<code>pushad</code> , selectores + <code>iret</code>	Guardado manual en <code>TrapFrame</code> (GPRs + <code>sepc/sstatus/scause/stval</code>) y <code>sret</code> .
PIT 8254 + IRQ0 + PIC <code>pic_finish1</code>	CLINT <code>mtimecmp/mtime</code> en M-mode → SSIP; <code>csrc sip</code> , SSIP.
Teclado PS/2 IRQ1 + <code>scancodes</code>	UART 16550 por polling; teclas ASCII.
VGA texto en 0xB8000	UART + secuencias ANSI (80×50 emulado).
TSS + cambio de tarea por hardware (<code>jmp <sel>:<off></code>)	<code>TrapFrame</code> por tarea + <code>sret</code> (cambio por software).
<code>int 0x2F (syscall)</code>	<code>ecall</code> : <code>a7=id</code> , <code>a0=arg/retorno</code> .
<code>invlpg</code> / recargar CR3	<code>sfence.vma</code> (por VA o global, opcionalmente por ASID).
<code>bochsrc</code> / Bochs	QEMU virt con <code>-bios none</code> .

Las referencias a instrucciones y CSRs siguen la convención de `~[1]` (ISA no privilegiada) y `~[2]` (ISA privilegiada, CSRs, Sv39).

4. Ejercicio 1 — Arranque y paso a modo supervisor

4.1. Qué hacía el TP x86

El TP3 original arrancaba en modo real desde el boot-sector, habilitaba A20, cargaba la GDT con `lgdt`, prendía PE de CRO, hacía el `jmp` lejano a modo protegido, cargaba los segmentos y fijaba el stack en `0x27000`. Además declaraba un segmento de pantalla aparte y limpiaba la VGA.

4.2. Equivalente en RISC-V

En RISC-V no hay segmentación ni A20. Los permisos de memoria salen de la PTE Sv39 (V, R, W, X, U, G, A, D), y lo más parecido a “pasar a modo protegido” es **ejecutar `mret` desde M-mode con `mstatus.MPP=1` (S-mode) y `mepc=kernel_main`**.

La cadena de arranque es:

1. QEMU salta a `_start` en `0x80000000` (M-mode).
2. `entry.S` inicializa `gp`, stack del kernel y limpia `.bss`, y llama a `machine_init`.
3. `machine.S::machine_init` (una “mini firmware” en M-mode, ver sección~4.4) configura PMP, delega traps a S-mode, programa el tick y ejecuta `mret` → `kernel_main` ya en S-mode.

4.3. `riscv/entry.S`

Fragmento relevante del entripoint:

```
_start:
    .option push
    .option norelax
    la gp, __global_pointer$
    .option pop

    csrw mie, zero
    csrci mstatus, 0x8          /* MIE */
    csrw sie, zero
    csrci sstatus, 0x2         /* SIE */

    la sp, kernel_stack_top

    /* Clear .bss */
    la t0, __bss_start
    la t1, __bss_end
1:
    bgeu t0, t1, 2f
    sb zero, 0(t0)
    addi t0, t0, 1
    j 1b
2:
    call machine_init
```

Detalles técnicos:

- `gp` y `.option norelax`. La psABI reserva `gp` para accesos *small-data* `gp`-relativos~[3]. Si no se desactiva la *linker relaxation* alrededor de `la gp, __global_pointer$`, el linker ve un `auipc/addi` y lo reescribe como un `addi gp, gp, 0 gp-relativo` — básicamente, asume que `gp` ya vale lo que tiene que valer. Resultado: `gp` queda en basura y cualquier acceso *small-data* posterior lee cualquier cosa. `.option norelax` apaga esa optimización en ese bloque puntual.

- **Stack del kernel.** Es el equivalente a “setear el stack en 0x27000” del TP x86. El símbolo `kernel_stack_top` está definido al final de un bloque en `.bss` de 8~KiB, resultado del linker (ver `riscv/linker.ld`). No hace falta una dirección mágica porque el linker coloca todo a partir de 0x80000000 y la ubicación concreta de el stack la decide el linker al resolver símbolos.

4.4. riscv/machine.S: mini firmware en M-mode

`machine_init` cumple el rol que en un sistema típico cumple OpenSBI (la firmware SBI de referencia en RISC-V~[4]). Como se corre con `-bios none`, no hay SBI, y el kernel tiene que proveer él mismo la configuración mínima de M-mode. El esqueleto sigue la receta *Bare Bones* para RISC-V documentada en OSDev~[11]. Los pasos:

1. **Stack y trap vector de M-mode.** `mscratch` guarda el tope de el stack de M-mode y `mtvec` apunta a `machine_trap_entry`.
2. **Delegación.** `mideleg` delega a S-mode los interrupts de supervisor (SSIP, STIP, SEIP) y `medeleg` delega las excepciones estándar 0..15. Así, las excepciones desde U-mode saltan directamente a `stvec` del kernel y no a `mtvec`.
3. **PMP.** Una sola región TOR que abarca todo el espacio (`pmpaddr0 = -1`, `pmpcfg0 = RWX+TOR`) le da a S-mode y U-mode acceso pleno según PMP. El control fino después lo dan las PTEs Sv39.
4. **Timer.** Se habilita MTIE en `mie` y se programa el primer `mtimecmp = time + g_timer_delta_ticks`.
5. **Salto a S-mode.** Se setea `mstatus.MPP=1 (S)`, `mstatus.MPIE=1`, `mepc=kernel_main` y se ejecuta `mret`. Esto es el análogo RISC-V del `jmp 0xA0:modo_protegido` del TP x86.

```
# Delegar interrupts a S-mode: SSIP, STIP, SEIP
li t0, ((1 << 1) | (1 << 5) | (1 << 9))
csrw mideleg, t0

# Delegar excepciones 0..15 a S-mode
li t0, 0xFFFF
csrw medeleg, t0

# PMP: R|W|X + TOR cubriendo 0..max
li t0, -1
csrw pmpaddr0, t0
li t0, 0x0F
csrw pmpcfg0, t0
```

4.5. “Segmento de pantalla” y limpieza inicial

Como no hay segmentación, la pantalla se resuelve por **mapeo de MMIO**: el UART 16550 de QEMU virt en 0x1000_0000. En cada page-table de tarea (y en la de idle) se mapea ese rango con una megapage (2~MiB) a nivel L1, con flags `V|R|W|A|D`, sin `U`, para que el kernel pueda escribirlo. El equivalente al “segmento de pantalla sólo accesible por el kernel” del TP x86 es entonces el bit `U=0` de esa entrada.

La limpieza y pintado del tablero equivalen, a nivel código, a `screen_draw / screen_drawBox` en `riscv/screen.cpp`. Internamente la pantalla se emula como un buffer `g_video[50][80]` de celdas (carácter + atributo estilo VGA), que se vuelca al UART mediante secuencias ANSI (`\x1b[?1049h` para el alternate screen, `\x1b[y;xH` para posicionar el cursor, `\x1b[fg;bgm` para atributos, etc.). Al imprimir, un atributo estilo `C_BG_RED | C_FG_WHITE` se traduce a los códigos ANSI 41 y 97.

5. Ejercicio 2 — Manejo de excepciones: IDT → stvec

5.1. Qué hacía el TP x86

Se armaba una IDT de 256 entradas. Para cada excepción (0..18) se daba de alta una `IDT_ENTRY` con el selector de código de kernel, `DPL=0`, `Present=1`, `Type=1110` (Interrupt Gate). Cada ISR (`_is0`, `_is1`, ...) se generaba con una macro en `isr.asm` e imprimía un mensaje. Se cargaba la IDT con `lidt`.

5.2. Equivalente en RISC-V

RISC-V no tiene una tabla indexada de gates: hay un único CSR, `stvec`, que apunta a la dirección a la que salta el procesador siempre que una excepción o interrupción tiene que atenderse en S-mode. El motivo lo decide el handler en software leyendo `scause`. Con lo cual:

- La IDT se reemplaza por una **rutina única en ASM** (`trap_entry`, en `riscv/trap.S`) que guarda todo el contexto y llama a un despachador en C++ (`trap_handle`, en `riscv/kernel.cpp`).
- “Selector + offset + DPL” ya no tiene sentido: `stvec` es una dirección, y el privilegio al que vuelve el handler sale de `sstatus.SPP`.
- El “cargar IDT con `lidt`” queda en:

```
asm volatile("csrw stvec, %0" : : "r"(trap_entry));
```

en `kernel_main`, con `stvec` en modo “Direct” (`MODE=0`). El modo “Vectored” (`MODE=1`) permitiría un salto por causa de interrupción; no se usa aquí.

5.3. Guardado de contexto: `TrapFrame` y `trap.S`

El equivalente de “pushad + selectores” es la estructura `TrapFrame` (`riscv/trap.h`):

```
struct TrapFrame {
    uint64_t ra, sp, gp, tp;
    uint64_t t0, t1, t2;
    uint64_t s0, s1;
    uint64_t a0, a1, a2, a3, a4, a5, a6, a7;
    uint64_t s2, ..., s11;
    uint64_t t3, t4, t5, t6;
    uint64_t sepc, sstatus, scause, stval;
};
```

`trap_entry` (en `riscv/trap.S`) hace, a grandes rasgos:

1. Cambia a una stack de trap dedicada (`trap_stack_top`). Esto es necesario porque, al entrar desde U-mode, `sp` apunta al stack del usuario, y usarlo implicaría ejecutar código de kernel en una página de usuario.
2. Guarda todos los GPRs (excepto `x0`) en el `TrapFrame` apuntado por `g_current_trapframe`.
3. **Recarga gp al del kernel** con `.option norelax`. Esta parte es crítica: los binarios de usuario se linkan en `0x08000000` y tienen su propio `__global_pointer$ ~0x08000800`, por lo que los accesos gp-relativos del kernel (pensados con el gp del kernel, `~0x8000...`) faltarían si el trap handler mantuviera el gp del usuario.
4. Guarda `sepc`, `sstatus`, `scause`, `stval`.
5. Llama a `trap_handle(tf)`, que devuelve un nuevo `TrapFrame*` (la “próxima tarea” a correr).

6. Restaura el contexto del TrapFrame retornado (incluyendo `sepc/sstatus`) y ejecuta `sret`.

La rutina `enter_user`, también en `trap.S`, se utiliza para el salto inicial desde `kernel_main` a la primera tarea U-mode: setea `g_current_trapframe`, carga directamente todos los registros desde el `TrapFrame` y ejecuta `sret`. Es el análogo RISC-V del `jmp 0xB8:0` del TP x86 para saltar a la tarea `idle`.

5.4. Probar una excepción

El TP x86 probaba la IDT forzando una división por cero. En este port, el binario de usuario `riscv/user/miner.c` provoca deliberadamente una excepción cuando el minero tiene `id 5`:

```
if (id == 5) {
    syscall_move(Forward); /* ... */
    *(volatile uint32_t*)0x0 = 0xDEADBEEF;
    for (;;) {}
}
```

Escribir en la dirección virtual `0x0` desde U-mode genera un *Store/AMO page fault* (`scause=15`) porque esa dirección no está mapeada. `trap_handle` mata a la tarea mediante `sched_kill()` y cede el `quantum` a `idle` (ver ejercicio~7).

6. Ejercicio 3 — Reloj, teclado e interrupción 47

6.1. Reloj: PIT + PIC → MTIP + SSIP

Qué hacía el TP x86

La `_isr32` llamaba a `nextClock` (animación del cursor) y `pic_finish1` (EOI al PIC maestro), y ejecutaba `iret`.

Equivalente en RISC-V

En RISC-V el timer está en CLINT/ACLINT[~][5]: dos MMIOs, `mtime` (contador libre) y `mtimecmp` (comparador por HART). Cuando `mtime ≥ mtimecmp`, se levanta **MTIP** (machine timer interrupt pending). El problema es que MTIP **solo** se puede atender en M-mode (no se puede delegar a S-mode directamente[~][2]). Un sistema con OpenSBI resolvería esto usando `sbi_set_timer`[~][4]; con `-bios none` no hay SBI y debemos resolverlo nosotros.

El patrón estándar (el mismo que usa xv6 en su build sin SBI[~][8] y que documenta el wiki OSDev para RISC-V[~][11]) es:

1. Atender el MTIP en M-mode (`machine_trap_entry`, `mcause=7`).
2. Reprogramar el próximo `mtimecmp`.
3. **Levantar un supervisor software interrupt (SSIP)** escribiendo `sip.SSIP=1` desde M-mode.
4. Ejecutar `mret` y dejar que la CPU dispare el handler S-mode (vía `stvec`) con `scause=1` (SSIP).

En `machine.S`:

```
# Reprogramar mtimecmp
csrr t0, time
la t1, g_timer_delta_ticks
ld t1, 0(t1)
add t0, t0, t1
csrr t1, mhartid
slli t1, t1, 3
li t2, MTIMECMP_BASE      # 0x0200_4000 + 8*hartid
add t2, t2, t1
sd t0, 0(t2)

# Levantar SSIP
li t0, 2
csrs sip, t0
```

Y del lado de S-mode, `trap_handle` reconoce SSIP como el tick:

```
if (is_interrupt) {
    if (cause_code == 1) { // SSIP
        const uint64_t ssip_mask = (1ull << 1);
        asm volatile("csrc sip, %0" : : "r"(ssip_mask));
        /* cooldowns + poll UART + spinner + scheduler */
        return schedule_next();
    }
    return tf;
}
```

El `csrc sip, SSIP` es el análogo de `pic_finish1`: limpia el bit pendiente para que el interrupt no se vuelva a disparar. La función `nextClock` del TP original (animación del spinner en la esquina) se preserva con `update_spinner()` en `kernel.cpp`.

6.2. Teclado: IRQ1 + scancodes → UART polling

Qué hacía el TP x86

La `_isr33` leía el scancode de `in al, 0x60`, pusheaba `eax` y llamaba a `idt_int_teclado` (que traducía el scancode a la acción del juego), luego `pic_finish1` y `iret`.

Equivalente en RISC-V

QEMU virt no tiene teclado PS/2 estándar; lo que hay es el UART 16550 sobre `-serial stdio`. Podría tomarse la interrupción del UART vía PLIC, pero para no sumar un mecanismo más al informe terminé haciéndolo por **polling en cada tick**, que a la frecuencia del reloj (~ 20 Hz) se percibe igual. En lugar de un ISR por tecla, `trap_handle` llama a `poll_uart_input()` al atender el tick, y ese drena el buffer con `uart::getc_nonblock()`:

```
int uart::getc_nonblock() {
    if ((mmio_read8(UART_LSR) & LSR_DATA_READY) == 0)
        return -1;
    return mmio_read8(UART_RHR);
}
```

Al llegar un byte se lo pasa a `handle_key`, que mapea teclas (no scancodes) a acciones de juego:

- Jugador ROJO: W (arriba), S (abajo), Q/E (spawnear minero).
- Jugador AZUL: I (arriba), K (abajo), O/P (spawnear minero).

El byte recibido se imprime en la esquina superior derecha en hex (equivalente al “mostrar la tecla 0..9 en pantalla” del TP x86), útil para validar input.

6.3. int 47 (x86) → ecall (RISC-V)

En x86 se declaraba una Interrupt Gate con `DPL=3` en la entrada 47 de la IDT, y la `_isr47` inicialmente seteaba `eax=0x42` y hacía `iret`. En RISC-V el análogo de esa *software interrupt* es `ecall`: desde U-mode genera un trap con `scause=8` (“Environment call from U-mode”). No hay un “número” configurable como en `int N`; lo que sí se mantiene del TP3 son los IDs de servicio, que en este port van por `a7` siguiendo la convención estándar de syscalls de RISC-V:

Servicio	ABI
<code>move(d)</code>	<code>a7=177788</code> , <code>a0=d</code>
<code>take()</code>	<code>a7=310311</code> , retorna en <code>a0</code>
<code>getId()</code>	<code>a7=760279</code> , retorna en <code>a0</code>

No se usa un “`eax` transitorio con `0x42`” porque el handler puede escribir directamente sobre `tf->a0` y el `sret` restaura `a0` como valor de retorno. El stub de usuario vive en `riscv/syscall.h`:

```
static inline int32_t syscall_take() {
    register uint64_t a7 asm("a7") = 310311;
    register uint64_t a0 asm("a0");
    asm volatile("ecall" : "=r"(a0) : "r"(a7) : "memory");
    return (int32_t)a0;
}
```

El despacho en `trap_handle` se describe en el ejercicio~7.

7. Ejercicio 4 — Paginación del kernel

7.1. Qué hacía el TP x86

`mmu_initKernelDir` armaba un Page Directory en `0x27000` y una PT en `0x28000`, con identity mapping de los primeros 4~MiB. Luego `kernel.asm` cargaba `CR3` con `KERNEL_PAGE_DIR` y seteaba `CR0.PG=1`.

7.2. Equivalente en RISC-V: Sv39

Sv39 es el esquema de paginación de 3 niveles más común en rv64:

- Direcciones virtuales de 39 bits (los superiores deben ser sign-extended).
- Tres niveles de page table con 512 entradas de 8 bytes cada una (4~KiB por tabla). VPN2 (bits 38:30), VPN1 (bits 29:21), VPN0 (bits 20:12).
- Páginas de 4~KiB, 2~MiB (*megapage*, leaf en L1) o 1~GiB (*gigapage*, leaf en L2).
- PTE de 64 bits con los bits V (1), R (2), W (4), X (8), U (16), G (32), A (64), D (128), seguidos de la PPN.

El equivalente conjunto de “`mov cr3, eax + or CR0, PG`” es escribir el CSR `satp` con formato:

$$\text{satp} = (\underbrace{\text{MODE}}_{\text{SV39}=8} \ll 60) \mid (\text{ASID} \ll 44) \mid \text{PPN}(\text{root})$$

seguido de un `sfence.vma` para invalidar caches de traducción. En `mmu.cpp`:

```
static inline uint64_t make_satp(uint64_t root_pa, uint16_t asid) {
    const uint64_t ppn = root_pa >> 12;
    return (SATP_MODE_SV39 << 60)
        | (static_cast<uint64_t>(asid) << 44)
        | (ppn & 0xFFFF'FFFF'FFFFull);
}
```

7.3. Identity mapping del kernel

Dado que en RISC-V rv64 el espacio virtual es holgado, en lugar de armar una PT nivel-0 que mapee los primeros 4~MiB como en el TP x86, se usa una única **leaf de 1~GiB en L2** para cubrir todo el rango de RAM relevante (`0x8000_0000..0xBFFF_FFFF`), con flags `V|R|W|X|A|D` (supervisor only, sin U):

```
static inline void map_kernel_gigapage(uint64_t* root) {
    const uint64_t va = 0x8000'0000ull;
    const uint64_t pa = 0x8000'0000ull;
    const uint64_t flags = PTE_V|PTE_R|PTE_W|PTE_X|PTE_A|PTE_D;
    root[vpn2(va)] = pte_make_leaf(pa, flags);
}
```

Esto reemplaza la tabla `KERNEL_PAGE_TABLE_0` del TP x86. Los bits A y D son requeridos por el hardware en el momento del primer acceso; setearlos *a priori* simplifica la implementación y está explícitamente permitido por la especificación [2].

7.4. UART como “MMIO del video”

El MMIO del UART 16550 se mapea como una **megapage** (2~MiB) en L1 con flags `V|R|W|A|D` (sin X ni U). Este bloque cumple el rol que en x86 cumplía el “segmento de pantalla”. El mapeo está presente tanto en la page-table del kernel como en la de cada tarea (aunque las tareas no tienen U en esa entrada, por lo que no pueden tocar el UART desde modo usuario).

7.5. Activación de paginación y PMP

Antes del `satp`, PMP tiene que autorizar el acceso a memoria en S-mode. Eso ya lo hizo `machine_init` en M-mode (sección~4.4). Por lo tanto, la “activación de paginación” del TP3 se reduce a una sola línea en `mmu_init`:

```
void mmu_init() {  
    if (!g_idle.initialized) { /* ... construye g_idle ... */  
        mmu_switch_to_idle(); // csrw satp + sfence.vma  
    }  
}
```

A partir de esa llamada, todas las direcciones en S-mode son traducidas por Sv39.

8. Ejercicio 5 — MMU de tareas

8.1. Qué hacía el TP x86

Este ejercicio era el más denso del TP3. Incluía:

- `mmu_init`: contadores `next_free_kernel_page` (desde `0x100000`) y `next_free_task_page` (desde `0x400000`).
- `mmu_initTaskDir`: construir el PD/PT de la tarea, identity-mapear kernel + área libre kernel, copiar el código original de la tarea (desde `MMU_BASE_ADDR_TAREAS`) a su celda física en el tablero, mapear `TASK_CODE` (`0x08000000`) a esas páginas con permisos U, y mapear `TASK_COMPARTIDA` a `g_shared[jugador]`.
- `mmu_mapPage` / `mmu_unmapPage`: helpers para mapear/desmapear una página virtual, invalidando el TLB con `tlbflush`.

8.2. Equivalente en RISC-V

La portación conserva el esquema “el código del minero vive físicamente en su celda del tablero”. Las funciones equivalentes viven en `riscv/mmu.cpp`:

- `mmu_task_init(slot, player, x, y)`: equivalente de `mmu_initTaskDir`.
- `mmu_task_move(slot, old_x, old_y, new_x, new_y)`: equivalente conjunto de “copiar 8~KiB + remapear `TASK_CODE`”.
- `mmu_switch_to_task(slot)` / `mmu_switch_to_idle()`: análogo a “cargar CR3” (escribir `satp`).

No se exponen `mmu_mapPage`/`mmu_unmapPage` genéricas, porque en este port el universo de mapeos es más restringido y conocido (kernel gigapage, UART megapage, dos páginas de código y dos páginas compartidas por tarea). El rol de esas funciones lo cumplen `map_user_code_pages` y `map_user_shared`, que actualizan directamente `t.l0_user[]`.

8.3. Allocator del kernel

El análogo del “contador de páginas libres” del TP x86 es un *bump allocator* a partir del final del `.bss`:

```
static uint64_t g_alloc_page = 0;

static void* alloc_page_table() {
    if (g_alloc_page == 0) {
        g_alloc_page = align_up_u64(
            reinterpret_cast<uint64_t>(&__bss_end), PAGE_SIZE);
    }
    void* page = reinterpret_cast<void*>(g_alloc_page);
    g_alloc_page += PAGE_SIZE;
    memset(page, 0, PAGE_SIZE);
    return page;
}
```

8.4. “Tablero como memoria”

El “área mapa” del TP3 (`0x400000..0x1C5FFFF`, 78×40 celdas de 8~KiB) se reemplaza por un **arreglo estático alineado a página**:

```

alignas(PAGE_SIZE) static uint8_t
    g_board[TABLERO_N_COLS * TABLERO_N_FILS * kCellBytes];

static inline uint64_t board_cell_pa(uint32_t x, uint32_t y) {
    const uint64_t ix = y * TABLERO_N_COLS + x;
    return reinterpret_cast<uint64_t>(&g_board[ix * kCellBytes]);
}

```

Esto evita tener que fijar direcciones físicas “mágicas” y deja que el linker resuelva el layout. Igualmente se cumple la propiedad del TP3: cada celda del tablero es un bloque contiguo de 8~KiB y `board_cell_pa(x,y)` es su dirección física.

Las áreas compartidas entre mineros del mismo jugador se resuelven también con arreglos estáticos:

```

alignas(PAGE_SIZE) static uint8_t g_shared[2][2 * PAGE_SIZE];

```

8.5. Construcción de la page-table de tarea

`mmu_task_init` construye la page-table completa de una tarea:

```

static inline void task_build_pagetable(TaskMmu& t,
    uint16_t asid, uint32_t player_id, uint64_t cell_pa) {
    t.root = reinterpret_cast<uint64_t*>(alloc_page_table());
    map_kernel_gigapage(t.root);
    ensure_l1_0(t);
    map_uart(t);
    map_user_code_pages(t, cell_pa);           // TASK_CODE (2 pags)
    map_user_shared(t, player_id);           // TASK_COMPARTIDA (2 pags)
    t.asid = asid;
    t.satp = make_satp(
        reinterpret_cast<uint64_t>(t.root), asid);
    t.initialized = 1;
}

```

El mapeo de `TASK_CODE` es el punto crítico de la mecánica del juego: a partir de la VA `0x08000000` se mapean dos páginas físicas contiguas a partir de `cell_pa`, con flags `V|R|W|X|U|A|D`. Es decir, *el minero puede leer, escribir y ejecutar en el mismo bloque de memoria* — exactamente la condición que hace posible que el juego copie el código de la tarea a otra celda y la tarea siga ejecutando desde `0x08000000` con código distinto.

8.6. `mmu_task_move`: movimiento del minero

Es el análogo a “copiar 8~KiB + `mmu_mapPage(TASK_CODE, ...) + tlbflush`” del TP x86:

```

void mmu_task_move(uint32_t slot,
                  uint32_t old_x, uint32_t old_y,
                  uint32_t new_x, uint32_t new_y) {
    TaskMmu& t = g_tasks[slot];
    if (!t.initialized) return;

    const uint64_t src_pa = board_cell_pa(old_x, old_y);
    const uint64_t dst_pa = board_cell_pa(new_x, new_y);

    memcpy((void*)dst_pa, (const void*)src_pa, kCellBytes);
    fence_i(); // ifetch coherence

    map_user_code_pages(t, dst_pa);
    sfence_vma_addr(kUserBase);
    sfence_vma_addr(kUserBase + PAGE_SIZE);
}

```

Detalles técnicos:

- `fence.i`. Después de escribir instrucciones en memoria, RISC-V no garantiza que el pipeline de instrucciones vea los bytes nuevos hasta una `fence.i` explícita [1]. En x86 la coherencia I\$/D\$ es automática; en RISC-V no, y sin esta instrucción el minero podría ejecutar el código viejo después del movimiento (me pasó debuggando: la tarea arrancaba, se “movía”, y seguía ejecutando como si estuviera en la celda original).
- `sfence.vma rs1=va, rs2=x0`. Invalida sólo la entrada del TLB para esa VA, el equivalente del `invlpg` de x86. Es más barato que un flush global y alcanza porque el único cambio fue `TASK_CODE`.

8.7. ASIDs

Sv39 permite incluir un ASID en `satp` para que el TLB no haga flush global al cambiar de address space. Este port asigna:

- `asid=0` para idle.
- `asid=slot+1` para mineros (1..20).

El `sfence.vma x0, x0` sólo se usa al inicializar una tarea; en runtime los cambios de address space se hacen con `mmu_switch()` que escribe `satp` (con su ASID) y hace un `sfence.vma` global (simplicidad > eficiencia).

9. Ejercicio 6 — Contexto de tarea: TSS → TrapFrame

9.1. Qué hacía el TP x86

El TP3 usaba **task switching por hardware**: un descriptor de TSS en la GDT por tarea (`tss_initGdtEntry`), una TSS por tarea (`fill_tss_idle`, `fill_tss`) con `eip`, `cr3`, `esp`, `eflags`, selectores de segmento, etc. El cambio de tarea se hacía con un `jmp <sel>:<off>` al selector de la TSS, dejando que el procesador guardara/cargara todo el contexto automáticamente.

9.2. Equivalente en RISC-V

RISC-V no tiene task switching por hardware, todo es por software. Se simplifica bastante: no hay GDT, ni TSS, ni descriptors; queda una estructura por tarea y nada más:

```
struct Task {
    TrapFrame tf;
    uint32_t initialized;
};
static Task g_tasks[20];
static TrapFrame g_idle_tf;
```

Cada `TrapFrame` cumple el rol que en el TP x86 cumplía la TSS: contiene el *snapshot* completo del estado de esa tarea. El campo `initialized` replica la noción de “descriptor presente”.

9.3. Inicialización de TrapFrame (equivalente de `fill_tss`)

Toda la “configuración inicial de la TSS” del TP x86 (`eip`, `esp`, `cs/ds/ss`, `eflags`) se colapsa en tres campos del `TrapFrame`:

```
static void init_user_trapframe(TrapFrame& tf, uint64_t entry_pc) {
    tf = {};
    tf.sepc = entry_pc;           // PC inicial
    tf.sp   = TASK_STACK;        // stack de usuario
    // SPIE=1 (SIE <- SPIE tras sret), SPP=0 (retorno a U-mode).
    tf.sstatus = (1ull << 5);
}
```

- `sepc=TASK_CODE (0x08000000) ⇔ tss.eip=dirección inicial del código.`
- `sp=TASK_STACK=TASK_CODE+2·4~KiB ⇔ tss.esp.`
- `sstatus.SPIE=1` asegura que `SIE=1` luego del `sret`, es decir, la tarea corre con interrupciones habilitadas (equivalente a `eflags.IF=1` del TP x86, donde se usaba `0x202`).
- `sstatus.SPP=0` indica que el `sret` devuelve el control a U-mode (equivalente a “DPL=3”). No hay “CS” ni “DS”; el modo es el único eje de privilegio.

Nótese que en RISC-V no existen “registros de segmento”: eso elimina por completo los campos `cs/ds/ss/fs/gs/es` de la TSS. Tampoco hace falta una “stack de nivel 0” por tarea (`ss0:esp0`): al entrar al kernel vía trap, se cambia a el stack de trap única (`trap_stack_top`), que alcanza porque `SIE` se limpia al entrar al trap (no hay traps anidados al mismo nivel de privilegio).

9.4. “Salto a la tarea Idle”

El TP x86 arrancaba con `jmp 0xB8:0` después de cargar `tr` con el selector de la tarea inicial. En este port, `kernel_main` hace:

```
init_idle_once();

asm volatile("csrw stvec, %0" : : "r"((uint64_t)trap_entry));
const uint64_t sie_mask = (1ull << 1) | (1ull << 5); // SSIE + STIE
asm volatile("csrs sie, %0" : : "r"(sie_mask));

TrapFrame* first = schedule_next();
enter_user(first);
```

`enter_user` (en `trap.S`) restaura todos los GPRs del `TrapFrame` y ejecuta `sret`. Con `sstatus.SPP=0` y `sepc=TASK_CODE`, esto transiciona a U-mode saltando al primer byte del binario de idle.

9.5. La tarea Idle

En el TP x86, idle era un binario del kernel en `0x00014000` que compartía `cr3` con el kernel. En este port idle es un binario de usuario *separado* (`riscv/user/idle.c`, ver sección ~12), con su propia address space (`asid=0`) y su propio buffer de 8~KiB como “celda” (no está en el tablero). Su código completo es:

```
void user_main() {
    for (;;) __asm__ volatile("nop");
}
```

El porqué de hacerla U-mode y no S-mode: mantiene el mismo ABI/path de entrada/salida que los mineros, simplifica el `TrapFrame` y evita tener que distinguir “contexto supervisor” vs “contexto usuario” en el scheduler.

10. Ejercicio 7 — Scheduler y servicios

10.1. Estructuras del scheduler

Las estructuras en `riscv/state.h` son un calco directo del TP3:

```

sched_miner_state_t sched_miners_state[20];
uint32_t sched_player_row[2];
uint32_t sched_miners_left[2];
uint8_t sched_last_miner[2];
uint8_t sched_this_miner;
```

La inicialización (`sched_init`) hace exactamente lo mismo que la versión x86: marca los 20 slots como muertos, reinicia contadores y filas. La única diferencia relevante es que, en este port, no hay una “GDT de TSS” que resetear: los slots son simples índices.

10.2. `sched_nextTask`

El algoritmo del TP3 (alternar entre jugadores y, si el opuesto no tiene mineros vivos, usar uno del jugador actual; si ninguno hay, usar `idle`) se preserva:

```

int32_t sched_nextTask() {
    uint32_t actual = (sched_this_miner < 10) ? PLAYER_A : PLAYER_B;
    uint32_t opuesto = (actual == PLAYER_A) ? PLAYER_B : PLAYER_A;
    uint16_t ix = 0;

    if (aux_get_next_alive_miner(opuesto, &ix)) {
        sched_this_miner = ix;
        sched_last_miner[opuesto] = ix % 10;
        return ix;
    }
    if (aux_get_next_alive_miner(actual, &ix)) {
        sched_this_miner = ix;
        sched_last_miner[actual] = ix % 10;
        return ix;
    }
    return -1; // idle
}
```

La diferencia clave con el informe x86 es el *tipo de retorno*: ya no es un selector de GDT, sino el índice 0..19 del minero (o `-1` para `idle`). Quien consume ese índice es `schedule_next` en `kernel.cpp`, que convierte el slot a `TrapFrame*` y hace `mmu_switch_to_task(slot)`:

```

static TrapFrame* schedule_next() {
    int32_t next = sched_nextTask();
    if (next < 0) return yield_to_idle();
    uint32_t slot = (uint32_t)next;
    init_task_if_needed(slot);
    mmu_switch_to_task(slot);
    g_current_slot = next;
    sched_this_miner = (uint8_t)next;
    return &g_tasks[slot].tf;
}
```

10.3. Despacho de ecall

La “rutina de int 47” del TP3 se vuelve un *branch* dentro de *trap_handle* cuando *scause=8*:

```
if (cause_code == 8) {           // ecall desde U-mode
    tf->sepc += 4;                // saltar la propia ecall
    poll_uart_input();
    const uint64_t sys_id = tf->a7;

    if (sys_id == 177788) {       // move(d=a0)
        game_move((e_direction_t)tf->a0);
        return yield_to_idle();  // equiv. del "jmp idle" del TP
    }
    if (sys_id == 310311) {       // take() -> a0
        tf->a0 = (uint64_t)game_take();
        return yield_to_idle();
    }
    if (sys_id == 760279) {       // getId() -> a0
        tf->a0 = (uint64_t)game_get_id();
        return tf;               // sin yield
    }
    if (g_current_slot >= 0) sched_kill();
    return yield_to_idle();
}
```

Detalles:

- *sepc += 4*: a diferencia de int N en x86 (que pushea la dirección *siguiente*), en RISC-V *sepc* apunta a la *ecall* misma. Hay que avanzarlo manualmente al retornar [2].
- **Yield a idle para move/take**. El TP3 requería que, tras un *move* o *take*, la tarea *Idle* ejecutara el resto del quantum. En x86 eso se implementaba con un *jmp <idle>* explícito. En este port se logra devolviendo *&g_idle_tf* como próximo *TrapFrame*: *trap.S* lo cargará y *sret* dejará a *idle* corriendo hasta el próximo SSIP.
- **getId sin yield**. También fiel al TP3: retorna al mismo minero con *tf->a0* seteado.
- **Syscall desconocido**. Se mata a la tarea (*sched_kill*), tal como en el TP original.

10.4. Intercambio de tareas por cada tick

El tick es un trap con *scause=1* (ver sección 6.1). La lógica es:

```
if (cause_code == 1) {
    asm volatile("csrc sip, %0" : : "r"((uint64_t)(1ull<<1)));
    if (g_spawn_cooldown_a) g_spawn_cooldown_a--;
    if (g_spawn_cooldown_b) g_spawn_cooldown_b--;
    poll_uart_input();
    update_spinner();
    return schedule_next();
}
```

schedule_next elige la próxima tarea, *trap_handle* retorna el *TrapFrame** correspondiente y *trap.S* se encarga de restaurarlo y saltar. Todo el “cambio de tarea” del TP3 (que en x86 era un *jmp <sel>:<off>*) termina siendo una *sret* con un *TrapFrame* distinto. No es particularmente elegante, pero funciona.

10.5. Atención de excepciones y desalojo

Cualquier excepción distinta de `ecall` mata a la tarea actual (si había una corriendo) y cede a `idle`:

```
// Any other exception: kill the current miner (if any) and keep going.
if (g_current_slot >= 0) {
    print(kHeaderExcPrefix, kHeaderExcX, kHeaderY,
          C_BG_BLACK | C_FG_LIGHT_RED);
    print_hex((uint32_t)cause_code, 2,
              kHeaderExcHexX, kHeaderY,
              C_BG_BLACK | C_FG_LIGHT_RED);
    sched_kill();
}
return yield_to_idle();
```

Esto cubre page faults (`scause` 12/13/15), illegal instructions (`scause`=2), misaligned loads/stores, etc. Por diseño, `scause` se imprime en hex en el banner superior: es el análogo reducido del “modo debug” del TP3.

10.6. Modo debug

El TP3 tenía un modo debug activable con la tecla y que mostraba el estado completo del procesador. En este port, la información se redujo a un indicador mínimo en la fila 0 del “video” (`TASK EXC XX`) con los dos dígitos hex del `cause_code`. El estado completo de registros no se imprime en pantalla; para debug profundo se usa QEMU (`-d int,guest_errors,cpu_reset,mmu`). Esta diferencia está documentada en la sección ~13.

11. Despacho de tareas y lógica del juego

La capa de “juego propiamente dicho” es, a nivel código, prácticamente idéntica al TP x86, porque ya era agnóstica a la ISA (punteros a estructuras, enteros, dibujos de pantalla). Los archivos `riscv/game.cpp` y `riscv/sched.cpp` conservan la lógica de `game_move`, `game_take`, `game_get_id`, `sched_spawn` y `sched_kill`.

Las dos diferencias relevantes respecto al informe x86:

- `game_move` ya no hace `copy_page` ni `mmu_mapPage` manuales: todo eso está encapsulado en `mmu_task_move` (sección 7.2). El cuerpo queda enfocado exclusivamente en la lógica del juego (cálculo del delta, wrap vertical, espejado para el jugador B, chequeo de base, puntaje, redibujo).
- `sched_spawn` no registra ningún descriptor de TSS en la GDT (no hay). El *caller* en `kernel.cpp::spawn_miner` se ocupa de:
 1. reservar el slot y marcarlo como vivo (responsabilidad de `sched_spawn`);
 2. resetear el `TrapFrame` del slot (`init_task_if_needed`); y
 3. llamar a `mmu_task_init(slot, player, x, y)`, que construye (o actualiza) la page-table y vuelca el binario del minero en la celda de `spawn`.

```
static void spawn_miner(uint32_t player_id) {
    int32_t slot = sched_spawn(player_id);
    if (slot < 0) return;
    const uint32_t u = (uint32_t)slot;
    g_tasks[u].initialized = 0;
    init_task_if_needed(u);
    const uint32_t x = sched_miners_state[u].pos_x;
    const uint32_t y = sched_miners_state[u].pos_y;
    mmu_task_init(u, player_id, x, y);
}
```

El *cooldown* de `spawn` (`g_spawn_cooldown_a/b`, 6 ticks) es un detalle UX agregado al port (evita spawnear 20 mineros apretando una tecla sostenida); no tiene contraparte en el TP3 original.

12. Tareas de usuario

El TP x86 tenía `taskA.c/taskB.c + idle.asm`, compilados como blobs que el kernel copiaba desde `0x10000` al tablero. Este port sigue el mismo espíritu, pero usa un *build separado* para los binarios de usuario:

- Fuentes: `riscv/user/miner.c`, `riscv/user/idle.c`, `riscv/user/entry.S`.
- Linker script propio (`riscv/user/linker.ld`) con `~=~0x08000000`.
- Producen `build/user_miner.bin / build/user_idle.bin` con `objcopy -O binary`.
- Se embeben en el kernel como símbolos `_binary_build_user_miner_bin_start / _end` vía `ld -r -b binary~[9]`. El kernel los copia a la celda correspondiente en `write_cell_image` durante `mmu_task_init`.

El `entry.S` de usuario es trivial:

```
_start:
    .option push
    .option norelax
    la gp, __global_pointer$
    .option pop
    call user_main
1: j 1b
```

El ABI para invocar servicios del sistema está en `riscv/syscall.h`, usando la convención `a7=id + a0=arg/retorno` (estándar *syscall* de Linux/RISC-V~[3]):

```
static inline void syscall_move(e_direction_t d) {
    register uint64_t a7 asm("a7") = 177788;
    register uint64_t a0 asm("a0") = (uint64_t)d;
    asm volatile("ecall" : "+r"(a0) : "r"(a7) : "memory");
}
```

El minero (`miner.c`) contiene dos modos:

- *Cosecha básica*: avanzar, tomar cristales, volver a base (análogo a la tarea ejemplo del TP).
- *Prueba de excepción*: cuando el minero tiene `id=5`, provoca una escritura en la VA `0x0` para generar un page fault (equivalente a la “división por cero” del TP3).

13. Limitaciones y diferencias respecto al TP3 original

- **Input por polling, no por IRQ.** El UART 16550 puede generar interrupts vía PLIC, pero se optó por polling en cada tick para minimizar la cantidad de mecanismos nuevos a documentar. A efectos de UX no hay diferencia perceptible con la frecuencia del tick (~ 20 Hz).
- **Display ANSI, no VGA.** No hay acceso a un framebuffer; el “tablero” es una emulación por UART. Esto implica cambios de render al redimensionar la terminal, y que el archivo de video no es un “segmento de pantalla” *stricto sensu*.
- **Modo debug simplificado.** Como se comentó en el ejercicio 7, la pantalla de debug completa del TP3 (registros, flags, stack) no se portó.
- **Sin OpenSBI.** Al correr con `-bios none`, el kernel implementa su propia firmware mínima (`machine.S`). Con `-bios default` (OpenSBI) el mismo kernel podría reemplazar `machine.S` por llamadas SBI (`sbi_set_timer`, `sbi_console_putchar`, etc.) sin cambios a la lógica en S-mode.
- **Single-hart.** QEMU virt soporta SMP, pero este port corre sobre un solo HART. Extenderlo a SMP requeriría IPI (via `msip`), serialización del scheduler, y probablemente per-hart scratch state.
- **Mapa de memoria distinto al del enunciado.** La figura del enunciado asume RAM desde `0x0000`. En QEMU virt la RAM arranca en `0x8000_0000`, así que las direcciones físicas concretas (kernel, page-tables, tablero) son otras; lo que se preserva es la *estructura* del layout.
- **3 niveles de page-table vs 2.** Sv39 tiene L2/L1/L0; el TP3 x86 (paginación 32 bits sin PAE) tenía PD/PT. Esto se aprovecha para mapear RAM con una sola gigapage, algo imposible en IA-32 de 32 bits sin PAE.

14. Build y ejecución

14.1. Dependencias

- `make`.
- Toolchain bare-metal: `riscv64-unknown-elf-{gcc, g++, as, ld, objcopy}`.
- `qemu-system-riscv64`.

14.2. Compilación

```
make -C riscv clean all
```

Genera `riscv/build/kernel.elf` (y `.bin`), que contiene embebidos los binarios de usuario (`build/user_miner.bin`, `build/user_idle.bin`).

14.3. Ejecución

```
make -C riscv run
```

que se expande a:

```
qemu-system-riscv64 -machine virt -nographic -monitor none \  
-serial stdio -bios none -kernel riscv/build/kernel.elf
```

14.4. Controles

- Jugador ROJO: W (arriba), S (abajo), Q o E (spawnear minero).
- Jugador AZUL: I (arriba), K (abajo), O o P (spawnear minero).

En la esquina superior derecha se muestra, en hex, el último byte recibido por el UART (útil para validar input).

Referencias

- [1] RISC-V International. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*.
<https://riscv.github.io/riscv-isa-manual/snapshot/unprivileged/>
- [2] RISC-V International. *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture*. Document version 20211203, ratificado en diciembre de 2021.
<https://github.com/riscv/riscv-isa-manual/releases/download/Priv-v1.12/riscv-privileged-20211203.pdf>
Versión snapshot en HTML: <https://riscv.github.io/riscv-isa-manual/snapshot/privileged>
- [3] RISC-V Non-ISA Working Group. *RISC-V ELF psABI Specification*.
<https://riscv-non-isa.github.io/riscv-elf-psabi-doc/>
- [4] RISC-V International. *RISC-V Supervisor Binary Interface Specification*.
<https://github.com/riscv-non-isa/riscv-sbi-doc>
- [5] RISC-V International. *RISC-V Advanced Core Local Interruptor (ACLINT) Specification*.
<https://github.com/riscv/riscv-aclint>
- [6] RISC-V International. *RISC-V Platform-Level Interrupt Controller (PLIC) Specification*.
<https://github.com/riscv/riscv-plic-spec>
- [7] QEMU Project. *QEMU System Emulation, 'virt' Machine (RISC-V)*.
<https://www.qemu.org/docs/master/system/riscv/virt.html>
- [8] Cox, R., Kaashoek, M.~F., Morris, R. *xv6: a simple, Unix-like teaching operating system (RISC-V port)*. MIT.
<https://github.com/mit-pdos/xv6-riscv>
<https://github.com/mit-pdos/xv6-riscv-book>
- [9] Free Software Foundation. *Using ld (GNU linker) — Binary input format*.
<https://sourceware.org/binutils/docs/ld/Options.html>
- [10] National Semiconductor. *PC16550D Universal Asynchronous Receiver/Transmitter with FIFOs*. Datasheet.
- [11] OSDev Wiki. *RISC-V* (página principal del wiki), *RISC-V Bare Bones*, y *RISC-V Meaty Skeleton with QEMU virt board*.
<https://wiki.osdev.org/RISC-V>
https://wiki.osdev.org/RISC-V_Bare_Bones
https://wiki.osdev.org/RISC-V_Meaty_Skeleton_with_QEMU_virt_board